

Java Fundamentals - Exercises

1. Classes and Objects

Assignment 1: Create a Book class with fields like title, author, price, and a method to apply discounts. Instantiate multiple objects and display their final prices.

Assignment 2: Design a BankAccount class with deposit, withdraw, and checkBalance methods. Track multiple account objects and simulate basic operations.

2. Constructors (Default, Parameterized)

Assignment 1: Create a Flight class with default and parameterized constructors. Include fields like flightNumber, destination, and duration. Create objects using both constructors.

Assignment 2: Create a Laptop class where the parameterized constructor sets brand, RAM, and processor. Use constructor overloading to provide flexibility.

3. Inheritance

Assignment 1: Create a Vehicle class and extend it with Bike and Car classes. Override a method like start() in each subclass.

Assignment 2: Build a Person class and extend it with Student and Teacher. Implement common methods and add role-specific behavior.

4. Polymorphism

a. Method Overloading

Assignment 1: Create a MathUtils class with overloaded methods calculate(int a, int b) for addition, and calculate(double a, double b) for multiplication.

Assignment 2: Create a Display class that overloads show() to print integers, strings, and arrays.

b. Method Overriding

Assignment 1: Create a base class Animal with makeSound() method. Override it in Dog, Cat, and Cow.

Assignment 2: Create a Payment class with method processPayment(). Override it in CreditCardPayment, UPIPayment, etc., to show different outputs.

5. Encapsulation

Assignment 1: Create a UserProfile class with private fields like username, email, and password. Use getters/setters and validate data (e.g., password length).

Assignment 2: Create a Circle class with private radius field and methods to calculate area/circumference. Use validation in setters.

6. Abstraction

a. Abstract Classes

Assignment 1: Create an abstract class Shape with abstract method area(). Implement it in Square and Triangle.

Assignment 2: Create an abstract class Appliance with methods turnOn() and turnOff(). Implement it in Fan, TV.

b. Interfaces

Assignment 1: Create an interface Printable and implement it in classes Document, Image, and Invoice.

Assignment 2: Define an interface Movable with method move(). Implement it in Car, Robot, and Drone.

7. Access Modifiers

Assignment 1: Create a class with all 4 access modifiers and test accessibility from:

- the same class
- a class in the same package
- a class in another package

Assignment 2: Create a package a with class A, and package b with class B. Try accessing protected/default members from B.

8. Static Members

Assignment 1: Create a Student class with static variable schoolName. Change it via one object and access through others.

Assignment 2: Create a Counter class that increments a static variable every time a new object is created. Display total instances created.

9. Keywords and Usage

a. super

Assignment 1: Use super to call a parent class constructor from a subclass.

Assignment 2: Use super to invoke a method from the parent class that has been overridden in the child class.

b. this

Assignment 1: Use this to resolve naming conflicts between instance variables and parameters.

Assignment 2: Use this() to call one constructor from another within the same class.

c. final

Assignment 1: Create a class with a final method and try to override it — observe compile-time error.

Assignment 2: Create a class with a final variable and show it cannot be reassigned.

10. Inner / Nested Classes

Assignment 1: Create a class Library with a non-static inner class Book. Show how to instantiate and access inner class from outer.

Assignment 2: Create a static nested class Logger inside a SystemUtils class. Use it to log actions in the main method.

11. Anonymous Classes

Assignment 1: Create an interface Greeting with method sayHello(). Use an anonymous class to implement and print a message.

Assignment 2: Create an abstract class Shape with method draw(). Use an anonymous class to provide its implementation and call draw()

Refer these for the mentioned topics in presentation::

Keywords:

// Parent class

```
class Employee {  
    final String companyName = "TechCorp"; // final variable  
  
    int empId;  
    String empName;  
  
    // Constructor  
    public Employee(int empId, String empName) {  
        this.empId = empId;    // this: refers to current object  
        this.empName = empName;  
    }  
  
    public void showDetails() {  
        System.out.println("Employee ID: " + empId);  
        System.out.println("Employee Name: " + empName);  
        System.out.println("Company: " + companyName);  
    }  
}  
  
// Child class
```

```
class Developer extends Employee {  
    static String department = "Software"; // static member shared  
    by all  
    String language;  
  
    // Constructor  
    public Developer(int empId, String empName, String language)  
    {  
        super(empId, empName); // super: refers to parent  
        constructor  
        this.language = language;  
    }  
  
    public void showDeveloperDetails() {  
        super.showDetails(); // Call parent method  
        System.out.println("Department: " + department);  
        System.out.println("Programming Language: " +  
this.language); // this: current instance  
    }  
}  
  
// Main class  
public class Company {  
    public static void main(String[] args) {  
        Developer dev1 = new Developer(101, "Alice", "Java");
```

```
        Developer dev2 = new Developer(102, "Bob", "Python");

        dev1.showDeveloperDetails();

        System.out.println("-----");

        dev2.showDeveloperDetails();

    }

}
```

InnerClass

// Outer class

```
public class Outer {
```

// Instance variable of outer class

```
String outerMessage = "Hello from Outer class!";
```

// Inner class

```
class Inner {
```

```
    public void displayMessage() {
```

```
        System.out.println("Hello from Inner class!");
```

```
        // Accessing outer class variable
```

```
        System.out.println("Accessing outerMessage: " + outerMessage);
```

```
    }
```

```
}
```

```
// Main method  
public static void main(String[] args) {  
    // Create object of Outer class  
    Outer outer = new Outer();  
  
    // Create object of Inner class using Outer class object  
    Outer.Inner inner = outer.new Inner();  
  
    // Call method of inner class  
    inner.displayMessage();  
}  
}
```

NestedClass

```
public class Outer {  
  
    // Static variable of outer class  
    static String outerMessage = "Message from Outer class";  
  
    // Static nested class  
    static class Nested {  
        public void display() {  
            // Can access static members of Outer class  
            System.out.println("Inside Nested class");  
        }  
    }  
}
```

```
        System.out.println("Accessing outerMessage: " + outerMessage);
    }
}
```

// Main method

```
public static void main(String[] args) {
    // Create object of static nested class directly
    Outer.Nested nestedObj = new Outer.Nested();
    nestedObj.display();
}
}
```

Anonymous

// Interface

```
interface Greeting {
    void sayHello();
}
```

// Main class

```
public class AnonymousClassExample {
    public static void main(String[] args) {

        // Anonymous class implementing Greeting interface
        Greeting greeting = new Greeting() {
```


@Override

public void sayHello() {

System.out.println("Hello from the anonymous class!");

}

};

// Call the method

greeting.sayHello();

}

}